Files
main
Go to file
ML4.1 metrics
ML4.2 threshold
ML4.3 cross-validation
img
README.md
ML4.4 diagnostics
ML4.5 hyperparameters
ML5.1 data integration
ML5.2 numeric features
ML5.3 categorical features
ML5.4 target
ML5.5 EDA
ML6.1 clustering
ML6.2 pca
ML7.3 text
ML7.4 images
misc
LICENSE

koroteevmv Исправлены битые ссылки на картинки 560ca87 · last year History

Name	Last commit message	Last commit date
..		
img	Картинки распределены по папкам	2 years ago
README.md	Исправлены битые ссылки на картинки	last year

README.md

Кросс-валидация

Цель работы

Познакомиться с основными типами разбиений данных при осуществлении кросс-валидации с использованием библиотеки sklearn.

Задания для выполнения

1. Загрузите датасет ирисы Фишера из библиотеки `sklearn.datasets`.
2. Сделайте hold-out разбиение данных. Для этого разделите данные на обучающую и валидационную выборки и выведите на экран соответствующие индексы разбиения.
3. Теперь сделайте разбиение перемешанных данных, зафиксировав воспроизводимость выбора данных после перемешивания, указав значение параметра `random_state=42` и выведите на экран соответствующие индексы разбиения.
4. Обучите модель логистической регрессии на обучающих данных. Выведите значения коэффициентов модели, полученных в результате обучения. Сделайте предсказание на тестовом наборе признаков. Выведите значение метрик `accuracy` и `f1-score`.
5. Разделите данные на обучающую и валидационную выборки по новому в соотношении 75-25. Обучите модель на этих данных, выведите значения получившихся коэффициентов модели. Выведите значения метрик и сравните их со значениями из предыдущего пункта. Сделайте вывод о том, влияет ли способ разбиения на результат.
6. Теперь сделайте k-блочную перекрёстную проверку модели (кросс-валидацию). Сравните полученные метрики с метриками, которые были при hold-out разбиении.
7. Теперь сделайте ту же самую перекрёстную проверку модели, используя библиотечную функцию `cross_val_score`. Убедитесь, что получится тот же результат.
8. Теперь сделайте k-блочную перекрёстную проверку модели (кросс-валидацию) со стратификацией. Проделайте всё тоже самое, что и в предыдущем пункте.
9. Теперь сделайте перекрёстную проверку, используя leave-one-out разбиение. Проделайте всё тоже самое, что и в предыдущем пункте.

Методические указания

Для данной лабораторной работы будем использовать известный датасет "Ирисы Фишера". Загружаем данные:

```
import pandas as pd
from sklearn.datasets import load_iris

iris = load_iris()
X=iris.data
y=iris.target
```

Выведите получившиеся данные на экран. Можно увидет, что данные хранятся в обычных массивах. Кроме самих данных в датасете присутствует дополнительная информация и описание данных. Познакомьтесь со структурой датасета.

Можно преобразовать данные для наглядности в DataFrame:

```
iris_data = pd.DataFrame(iris['data'], columns=iris['feature_names'])
name_map = {0: 'Iris-setosa', 1: 'Iris-versicolor', 2:'Iris-virginica'}
iris_data['class'] = [name_map[k] for k in iris['target']]
iris_data.head(10)
```

index	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	class
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa
5	5.4	3.9	1.7	0.4	Iris-setosa
6	4.6	3.4	1.4	0.3	Iris-setosa
7	5.0	3.4	1.5	0.2	Iris-setosa
8	4.4	2.9	1.4	0.2	Iris-setosa
9	4.9	3.1	1.5	0.1	Iris-setosa

Тестовая выборка

Для более правильного оценивания эффективности работы моделей разобьем исходную выборку на две части: тренировочную и тестовую:

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.15, random_state=5)
```

В данном случае мы резервируем 15% данных для тестовой выборки.

Обучим модель логистической регрессии и выведем значения метрик:

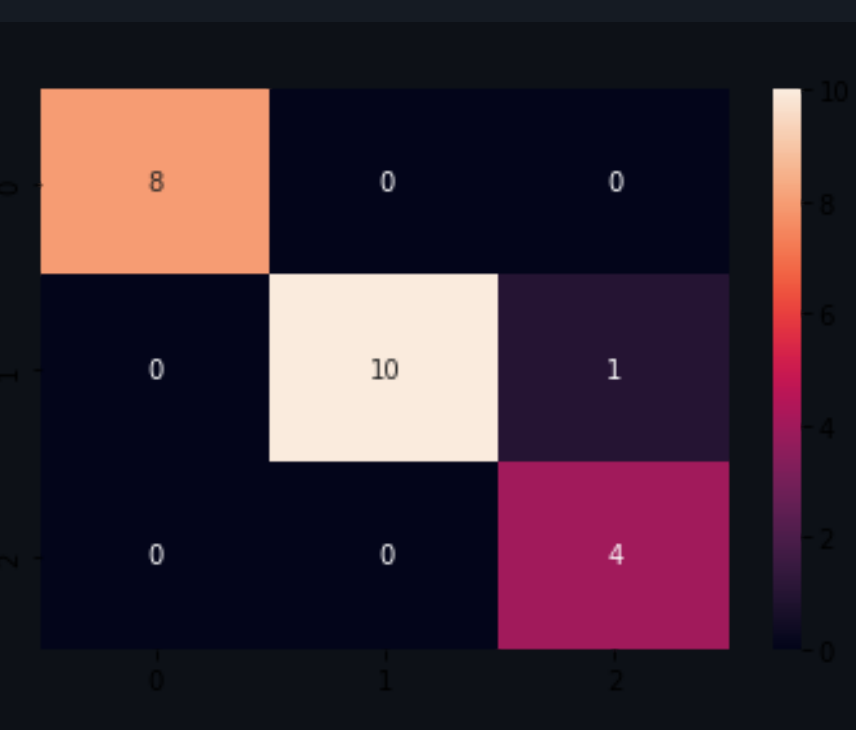
```
from sklearn.metrics import accuracy_score, f1_score
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(solver='liblinear')
model.fit(X_train, y_train) #Обучение трейновой выборке
y_pred = model.predict(X_test) #Предсказание для тестовой выборки
print(accuracy_score(y_test, y_pred))
print(f1_score(y_test, y_pred, average='macro'))
```

```
0.9130434782608695
0.9074074074074074
```

Видим, что модель обучилась до уровня точности 91%. Насколько это хорошо, судить сложно, надо сравнивать с результативностью других моделей. Для более наглядного представления результата можно вывести отчет о классификации:

```
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True)
```

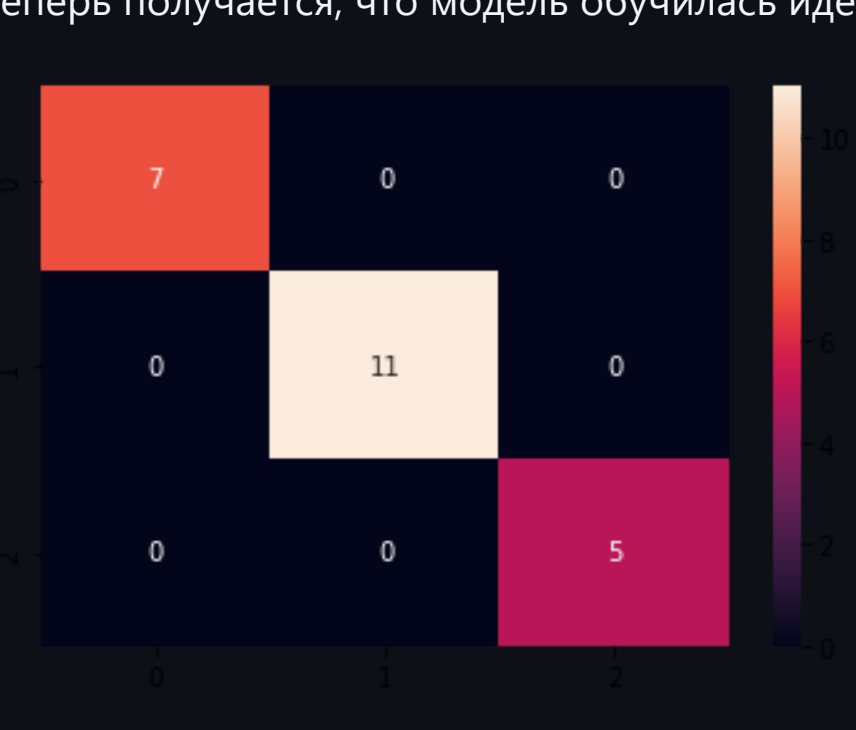


Но заметим, что эта оценка производилась именно при данном разбиении. Что будет, если мы сделаем другое разбиение датасета на две части? Давайте повторим разбиение с другим значением *random_state*, обучим другую модель и выведем те же метрики:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.15, random_state=0)
model = LogisticRegression()
model.fit(X_train, y_train) #Обучение трейновой выборке
y_pred = model.predict(X_test) #Предсказание для тестовой выборки
print(accuracy_score(y_test, y_pred))
print(f1_score(y_test, y_pred, average='macro'))
```

```
1.0
1.0
```

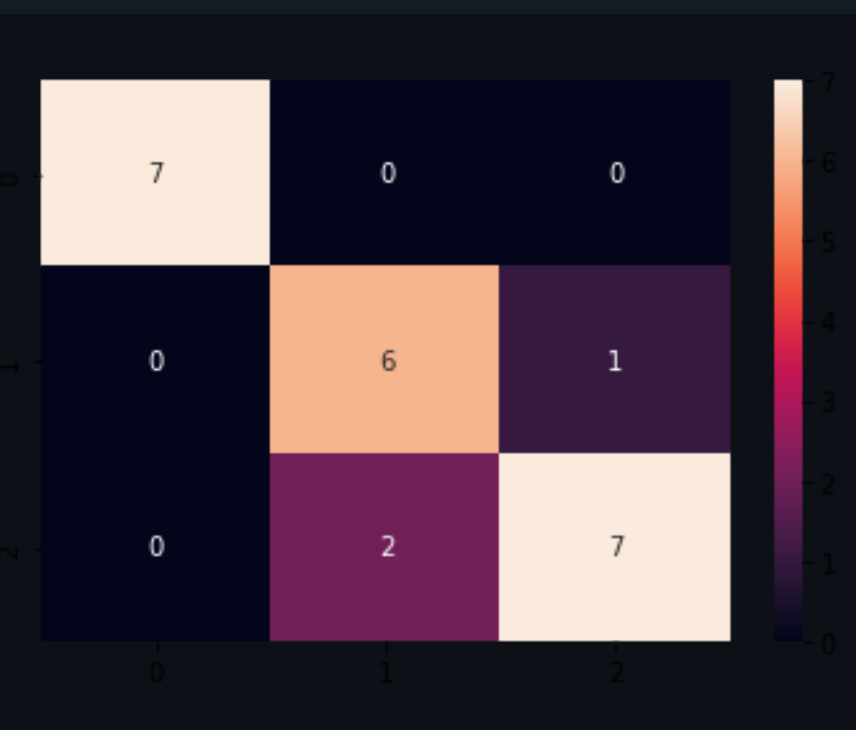
Теперь получается, что модель обучилась идеально. Это же подтверждает и матрица:



Можно повторять эту процедуру несколько раз и каждый раз будут получаться разные значения метрик. Например при таком разбиении модель обучается гораздо хуже:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.15, random_state=8)
model = LogisticRegression()
model.fit(X_train, y_train) #Обучение трейновой выборке
y_pred = model.predict(X_test) #Предсказание для тестовой выборки
print(accuracy_score(y_test, y_pred))
print(f1_score(y_test, y_pred, average='macro'))
```

```
0.8695652173913043
0.8745098039215686
```



Для устранения этих случайных колебаний и нужна перекрестная проверка или кросс-валидация.

Перекрестная проверка

Оценим работу построенной модели с помощью перекрёстной проверки.

Импортируем нужные библиотеки:

```
from sklearn.model_selection import KFold,StratifiedKFold,LeaveOneOut, cross_val_score
```

k-fold разбиение

В k-блочной перекрёстной проверке исходные данные разбиваются на k (примерно) равных по количеству частей, называемых "блоками", на $k-1$ из которых производится обучение, а на 1 валидация. В результате получается более робастная оценка эффективности выбранной модели.

Создаём k-блочное разбиение (KFold):

```
kf = KFold(n_splits = 3,shuffle=True, random_state=15)
kf
```

Метод `split()` - возвращает индексы разбиения:

Сделаем разбиение на блоки:

```
for i, (train_index, test_index) in enumerate(kf.split(y)):
    print("Fold {}: Длинна train: {}, Длинна test: {}".format(i+1, len(train_index), len(test_index)))
    print('Train: index={}\n Test:  index={}'.format(train_index, test_index))

    X_train, X_test = X[train_index], X[test_index]
    y_train, y_test = y[train_index], y[test_index]
```

Сделаем кросс-валидацию:

```
metrics_accuracy = []
metrics_f1 = []
model = LogisticRegression(solver='liblinear')
for i, (train_index, test_index) in enumerate(kf.split(y)):
    X_train, X_test = X[train_index], X[test_index]
    y_train, y_test = y[train_index], y[test_index]
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    metrics_accuracy.append(accuracy_score(y_test, y_pred))
    metrics_f1.append(f1_score(y_test, y_pred, average='macro'))
```

Выведем соответствующие массивы с метриками:

```
print('Значения метрики accuracy: {} \nЗначения метрики f1: {}'.format(metrics_accuracy, metrics_f1))
```

Выведем среднее значение метрики:

```
import numpy as np
print("Среднее по кросс-валидации: ", np.array(metrics_f1).mean())
```

Это и будет наша кросс-валидированная оценка метрики. Она гораздо ближе к истинному уровню эффективности модели за счет того, что все случайные ошибки выборки усредняются. Можно еще вывести дисперсию данной оценки, которая показывает степень уверенности в ней:

```
import numpy as np
print("Среднее по кросс-валидации: ", np.array(metrics_f1).mean())
```

Выполняем кросс-валидацию с помощью функции cross_val_score:

```
cv_results = cross_val_score(model,                  # модель
                             X,                      # матрица признаков
                             y,                      # вектор цели
                             cv = kf,                # тип разбиения (можно указать просто число фолдов cv = 3)
                             scoring = 'accuracy',   # метрика
                             n_jobs=-1)              # используются все ядра CPU

print("Кросс-валидация: ", cv_results)
print("Среднее по кросс-валидации: ", cv_results.mean())
print("Дисперсия по кросс-валидации: ", cv_results.std())
```

Этот код делает примерно то же, что и предыдущий, но автоматически, с использованием библиотечной функции.

```
Кросс-валидация:  [1.   0.94 0.94]
Среднее по кросс-валидации:  0.96
Дисперсия по кросс-валидации:  0.028284271247461926
```

Валидация по K-блокам (фолдам) - это самый распространенный алгоритм разбиения датасета на блоки. Он является золотым стандартом в научных исследованиях.

Stratified k-Fold

Метод stratified k-Fold — это метод k-Fold, использующий стратификацию при разбиении на фолды: каждый фолд содержит примерно такое же соотношение классов, как и всё исходное множество. Такой подход может потребоваться в случае, например, очень несбалансированного соотношения классов.

Создаём стратифицированное k-блочное разбиение (StratifiedKFold):

```
skf = StratifiedKFold(n_splits=3,shuffle=True, random_state=15)
skf.get_n_splits(X, y)
```

Выведем разбиение на блоки:

```
for i, (train_index, test_index) in enumerate(skf.split(X,y)):
    print(f"Fold {i+1}:")
    print('Train: index={}\n Test:  index={}'.format(train_index, test_index))

    X_train, X_test = X[train_index], X[test_index]
    y_train, y_test = y[train_index], y[test_index]
```

Выполняем кросс-валидацию с помощью функции cross_val_score:

```
cv_results = cross_val_score(model,                  # модель
                             X,                      # матрица признаков
                             y,                      # вектор цели
                             cv = skf,           # тип разбиения
                             scoring = 'f1_macro',   # метрика
                             n_jobs=-1)              # используются все ядра CPU

print("Кросс-валидация: ", cv_results)
print("Среднее по кросс-валидации: ", cv_results.mean())
```

Leave-one-out

Метод leave-one-out (LOO) является частным случаем метода k-Fold: в нём каждый фолд состоит ровно из одного семпла.

Создаём разбиение:

```
loo = LeaveOneOut()
```

Сделаем разбиение на блоки:

```
for i, (train_index, test_index) in enumerate(loo.split(X)):
    print(f"Fold {i+1}:")
    print('Train: index={}\n Test:  index={}'.format(train_index, test_index))

    X_train, X_test = X[train_index], X[test_index]
    y_train, y_test = y[train_index], y[test_index]
```

```
cv_results = cross_val_score(model,                  # модель
                             X,                      # матрица признаков
                             y,                      # вектор цели
                             cv = loo,           # тип разбиения
                             scoring = 'f1_macro',   # метрика
                             n_jobs=-1)              # используются все ядра CPU

print("Кросс-валидация: ", cv_results)
print("Среднее по кросс-валидации: ", cv_results.mean())
```

Контрольные вопросы

1. Зачем нужно применять кросс-валидацию?
2. В чём заключается процесс кросс-валидации?
3. В чем достоинства и недостатки каждого метода кросс-валидации?
4. Какой метод кросс-валидации можно применять на данных с большим дисбалансом классов?
5. Можно ли бороться с недообучением при помощи кросс-валидации? А с переобучением?
6. Какие основные типы разбиений данных используются при кросс-валидации?
7. Какой тип кросс-валидации можно применять есть нужно сделать очень большое количество проходов?

Дополнительные задания

1. Изучите разбиение Leave-P-Out. Продемонстрируйте работу этого алгоритма на примере из лабораторной работы.
2. Изучите функцию cross_validate(). Продемонстрируйте работу этой функции на тех же данных.
3. Оцените при помощи кросс-валидации другие метрики эффективности для этой же модели.
4. Сравните кросс-валидированные результаты работы нескольких моделей на одних и тех же данных.
5. Повторите анализ на другом датасете: встроенном наборе данных о диабете.
6. Сделайте k-блочную перекрёстную проверку (KFold) модели логистической регрессии, предварительно стандартизировав данные. Для этого создайте конвейер с помощью `make_pipeline` из библиотеки `sklearn.pipeline`, который будет стандартизировать, а затем выполнять логистическую регрессию.